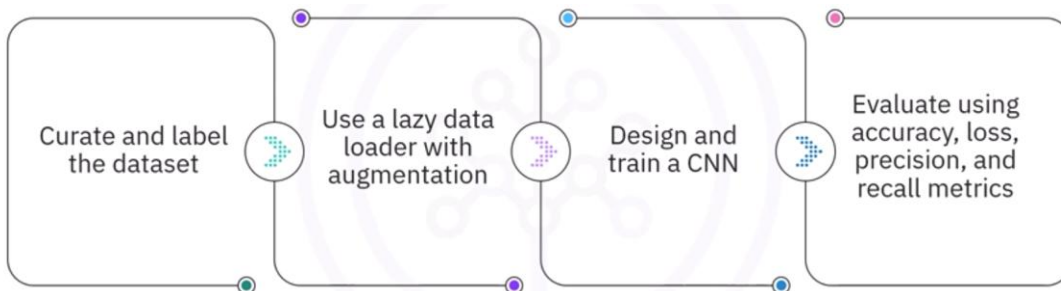


Why CNNs for Image Classification

- More robustness and accuracy than a manually defined set of rules or transitional machine learning pipelines

Steps to Create a CNN Classifier



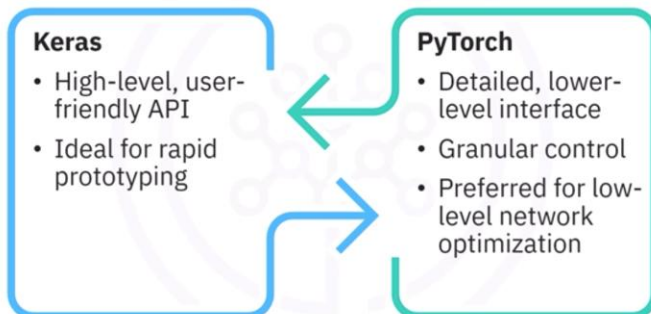
Frameworks: Keras and Pytorch

Uses of the frameworks

- Handling low-level tensor ops, gradient calculations, and GPU management
- Focusing on architecture and experimentation
- Embedding optimization:
 - Efficient GPU kernels
 - Automatic differentiation
 - Mixed-precision arithmetic
 - Distributed training routines

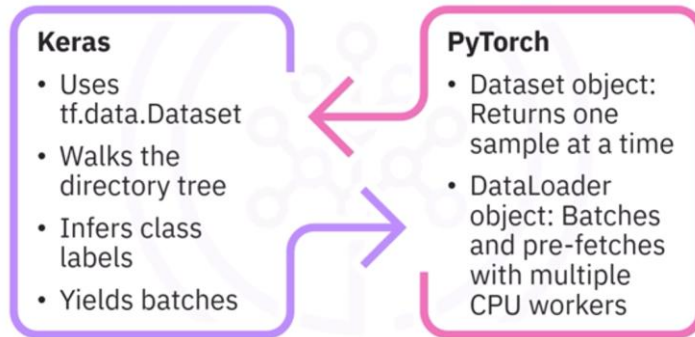
Implementation of CNNs

- Both can create a CNN capable of labelling images



Data Loading

- Both use lazy loading
- Keras uses `tf.data.Dataset`
- Pytorch uses Dataset object and DataLoader



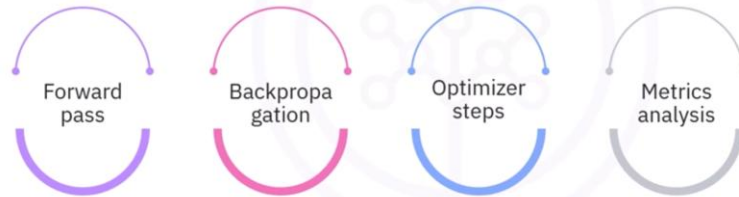
Data Augmentation

- Both use on-the-fly augmentation (meaning you never store augmented copies on disks), and the model sees a fresh rendition of each image during every epoch
- Keras offers preprocessing layers and the image data generator
- Pytorch uses the `torchVision.transforms` module

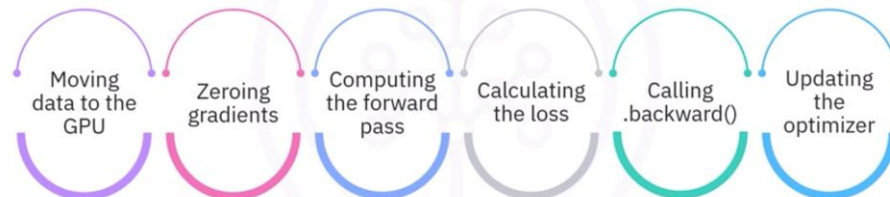


Model Architecture

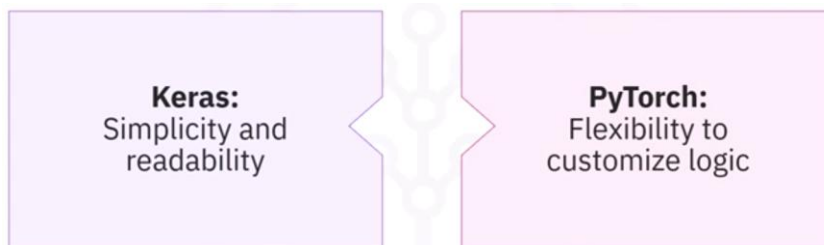
- Keras
 - Streamlines CNN construction using **sequential** API
 - Manages the entire training loop via the **model.fit** method (which contains the forward pass, back propagation, optimizer steps, and metrics analysis)



- Pytorch
 - Architecture defined as a **subclass of nn.Module**
 - The training loop must be **implemented explicitly** (includes moving data to the GPU, zeroing gradients, computing the forward pass, calculating the loss, calling `.backward()`, and updating the optimizer)



- Summary
 - Keras: concise, high-level interface that minimizes code
 - PytorchL hands-on, transparency



Evaluation Metrics for CNNs

Accuracy

- Proportion of correct predictions

Precision

- $\frac{\text{True positives}}{(\text{True positives} + \text{False positives})}$

Recall

- Measures sensitivity
- $\frac{\text{True positives}}{(\text{True positives} + \text{False negatives})}$

F1 score

- Harmonic mean of precision and recall

- **Accuracy** can be misleading for skewed datasets
- **Precision**: of all positive predictions, how many were correct?
- **Recall**: how many were correctly identified?
- **IoU** (intersection over union): overlap between predicted and ground truth bounding boxes)
- **mAP** (mean average precision): avg precision across classes

Confusion matrix

- Tabular summary of:
 - True positives
 - False positives
 - True negatives
 - False negatives
- Reveals confusions in classes

ROC and AUC

- Trade-off between true-positive and false-positive rates

IoU and mAP:

- IoU
 - Measures overlap in bounding boxes
- mAP
 - Average precision across all classes

- Keras and Pytorch

Keras:

- Pass metrics into the compile step
- Log them during fit

PyTorch:

- Calculate metrics using:
 - scikit-learn
 - torchmetrics

Fair Comparison

- Ensure matching batch sizes, learning rates, optimizer choices, number of epochs, weight initialization schemes, and softmax layer placement
- Keras typically includes softmax inside the model
- Pytorch omits softmax and feeds raw logits into the loss function

Jane, an AI Engineer at GreenField Innovations, is implementing a CNN model using PyTorch to classify images of agricultural fields. She needs to evaluate the model's performance. Which metric should she use to assess the model's ability to distinguish between classes?

Cross-entropy loss

Model accuracy

AU-ROC (Area Under Receiver Operating Characteristic curve)

Mean Squared Error

AU-ROC is a performance measurement for classification problems at various threshold settings. It tells how much the model is capable of distinguishing between classes.

At FarmTech Inc., Liam is constructing a CNN model using PyTorch to classify images of different crop types. He needs to ensure the model's architecture is suitable for the task. What is a key component he should include in the CNN model?

Recurrent layers to capture temporal sequences

Dropout layers to increase model capacity

Batch normalization layers to reduce model size

Convolutional layers to extract spatial features

Convolutional layers are fundamental in CNNs for extracting spatial hierarchies in images, which is crucial for image classification tasks.

At AgriTech Solutions, the data science team is tasked with classifying satellite images of agricultural land. They plan to build a CNN classifier using Keras. Which of the following steps should they take first in this process?

Train the CNN model using the preprocessed dataset.

Compile the CNN model with an appropriate loss function and optimizer.

Initialize the CNN model architecture with layers.

Preprocess the dataset to ensure all images are of the same size.

Preprocessing the dataset is crucial to ensure consistency in input size for the CNN model, which is a foundational step before model building.

Why is HeUniform() passed as kernel_initializer to many convolutional and dense layers?

It freezes pretrained ImageNet weights.

It schedules the learning rate automatically.

It initializes weights suited to ReLU-based activations.

It regularizes the model with L2 norms.

HeUniform initializer samples weights to keep variance constant through ReLU layers, promoting stable training.

Inside the evaluation (validation) loop, why is torch.no_grad() used?

To enable GPU acceleration

To reset model weights

To randomize batch order

To disable gradient calculation, saving memory and computation

Gradients aren't needed at inference time; turning them off speeds up evaluation and lowers memory use.

Why might F1 Score be preferred over accuracy on highly imbalanced datasets?

It balances precision and recall, focusing on minority-class performance.

It considers only true positives and true negatives.

It is threshold-independent like ROC-AUC.

It measures the area under the precision-recall curve.

F1 is the harmonic mean of precision and recall, ensuring that both false positives and false negatives are taken into account even when class counts are unequal.

How can you generate a detailed text summary containing per-class precision, recall, and F1?

`confusion_matrix`

`roc_curve`

`accuracy_score`

`classification_report` from the `sklearn.metrics` module

`classification_report` prints precision, recall, and F1, and supports each class and their macro and weighted averages.